



Felix Hawranek: 11946342

Moritz Pik: 12131711



Documentation

Seminar Paper

Supervisor

DI Johannes Zmölnig

Graz, February 2026

Contents

1. Getting Started	3
1.1. Installation	3
1.2. Running a program	3
2. Syntax	4
2.1. Syntax Basics and Program Structure	4
2.1.1. BÆNG Language Basics	4
2.1.2. BÆNG-JSON Basics	5
2.2. Supported statements and Commands	5
2.2.1. Assignments	5
2.2.2. Function calls	6
2.2.3. Conditionals	6
2.2.4. Loops	6
2.2.5. Console output	7
2.2.6. Impulse Response Manipulation	7
2.2.7. Exporting the Impulse Response	8
2.3. Expressions	9
2.3.1. Numerical Expressions	9
2.3.2. Boolean Expressions	9
2.3.3. Operator Precedence and Parentheses	10
2.3.4. String Expressions	10
2.3.5. Built-in Functions	10
2.4. Conditions	10
3. Functions and Execution Model	11
3.1. Overview	11
3.2. Function Definition and Parameters	11
3.3. Iteration Over the Impulse Response	11
3.4. Side Effects and Data Modification	11
3.5. Function Calls and Execution Order	11
4. Complete Example	12
4.1. Initialization of the Impulse Response	12
4.2. Room Simulation Function	13
4.3. Recursive Reflections Function	14
4.4. Running the Simulation	15
5. General Coding Conventions	16

1. Getting Started

1.1. Installation

To get started with BÆNG, you need to install the interpreter and parser. Both are provided as Python files in the Git repository. Python 3.11 or newer is required to run.

- (1) Follow the official Python guide for your platform via <https://www.python.org/downloads/>.
- (2) Clone the BÆNG repository:

Cloning Git repository

Terminal 1.1

```
git clone https://github.com/FHawranekGit/baeng.git
cd baeng
```

- (3) Overview of the installed files:
 - `README.md` - gives an overview of the language.
 - `baeng.py` - the main interpreter for executing BÆNG scripts.
 - `baengParser.py` - the parser module used to parse BÆNG code into an executable format.
 - `requirements.txt` - holds the dependencies.
 - `simple_example.baeng` - an executable example script.
 - `complete_example.baeng` - an executable example script that creates a simple reverb impulse response.
- (4) Install the needed dependencies:

Installing requirements.txt

Terminal 1.2

```
pip install -r requirements.txt
```

You are ready to go now and can run your first program.

1.2. Running a program

To run the example program, simply run the interpreter using `example.baeng` as an argument.

Running example.baeng

Terminal 1.3

```
python baeng.py example.baeng
```

The example program initializes an impulse response with a length of one second and a sample rate of 48 kHz. The first sample of this IR is set to a value of 1 by using the `setSample()` function. At the end of the program, the modified impulse response is automatically saved as `output.wav`.

simple_example.baeng

BÆNG 1.4

```
set IR: (48000, 1.0, "output.wav")
setSample(0, 1)
```

2. Syntax

2.1. Syntax Basics and Program Structure

2.1.1. BÆNG Language Basics

In this syntax, code structure is defined by indentation using four spaces rather than explicit block delimiters. Statements do not end with semicolons, and each line contains only a single statement, which helps keep the code clear and readable. Consistent whitespace is therefore syntactically significant, as indentation determines code hierarchy and defines the scope of functions and control structures. All array or sample indexing in BÆNG starts at 0.

A BÆNG program can be roughly structured in three parts. Every script starts with the initialization of an impulse response, with a given length `t` (of type `int` or `float`), a sample rate `fs` (of type `int`), and a path to where the output file should be stored (of type `string`). This sets up an IR object that can be referenced and manipulated throughout the program.

IR initialization

BÆNG 2.1

```
set IR: (fs, t, "path/to/filename.wav")
```

After initializing the impulse response, user functions can be defined. User functions are functions defined by the user to encapsulate reusable logic within a program. Unlike built-in functions provided by the language, a user function is written to perform a specific task required by the application and can be called multiple times from different parts of the program. A user function consists of a function name `foo`, optional parameters that receive input values `bar`, and a block of indented code that defines the behavior executed when the function is called. Parameters allow the function to operate on different inputs, while the function body contains the instructions that implement the desired operation. In BÆNG, every user function is executed iterating over the IR object. This and more information about user function behavior is described in Section 3 (see page 11).

User function definitions

BÆNG 2.2

```
func foo(bar1, bar2):  
    indented code
```

After user functions are defined, the main code section contains the sequence of statements that form the primary execution flow of the program. This section is evaluated after initialization and function declarations, and it typically consists of calls to user functions, parameter assignments, and operations that modify or generate data within the impulse response. The main code coordinates how previously defined functions are applied, determining the order and conditions under which processing occurs. Because functions in BÆNG operate by iterating over the IR object, the main code primarily specifies what operations are applied rather than how iteration is performed internally. Keeping execution logic in the main section separate from function definitions improves readability and allows reusable behavior to remain isolated from program-specific control flow.

2.1.2. BÆNG-JSON Basics

Every BÆNG program is internally converted into a JSON representation before execution. This intermediate format is used by the interpreter to organize program structure in a consistent and machine-readable way. While users typically write programs using the standard BÆNG syntax, it is also possible to author programs directly in JSON if preferred.

In this representational example, the impulse response configuration is stored in the `IR` field as an array containing the sample rate, duration, and output file path. User-defined functions are represented as objects whose names correspond to the function names in the original program. Each function specifies its input parameters in a `PARAMS` array and its executable instructions in a `CODE` array. The main execution flow of the program is likewise stored in a top-level `CODE` array, which defines the sequence of operations performed after the initialization.

structure.json

BÆNG-JSON 2.3

```
{
  "IR": [48000, 1.0, "output.wav"],
  "foo": {
    "PARAMS": ["bar1", "bar2"],
    "CODE": [ /* user function code */ ]
  },
  "CODE": [ /* main code */ ]
}
```

This structure allows the interpreter to separate initialization data, reusable function logic, and the main execution sequence while maintaining a clear and predictable mapping between BÆNG syntax and its executable form.

2.2. Supported statements and Commands

The following statements form the core executable elements of the language. Each statement performs a specific operation and contributes to the overall processing of the impulse response. Statements are evaluated sequentially unless control flow constructs alter execution order.

2.2.1. Assignments

Assignment

BÆNG 2.4

```
a = expr
```

Assignment JSON equivalent

BÆNG-JSON 2.5

```
["define", "a", "expr"]
```

The assignment statement evaluates the expression `expr` and binds its resulting value to the variable `a`. If `a` has not been previously defined, it is created automatically at the point of assignment. The language uses dynamic typing, meaning that variables are not bound to a fixed data type and may hold values of different types during program execution. Reassigning a variable replaces the previously stored value. Assignments are commonly used to store intermediate results, configure parameters, or prepare values for later use in function calls.

2.2.2. Function calls

Function call

BÆNG 2.6

```
foo(bar1, bar2)
```

Function call JSON equivalent

BÆNG-JSON 2.7

```
["foo", ["bar1", "bar2"]]
```

A function call invokes the user-defined function `foo` and passes the provided arguments as input parameters. The arguments are evaluated before the function is executed. Function calls allow reusable logic to be applied consistently across different parts of a program and help separate processing logic from execution flow. In BÆNG, user functions operate over the impulse response context, meaning their internal operations are applied during iteration over the IR as defined by the runtime. Function calls may produce side effects such as modifying samples or updating variables depending on the function implementation.

2.2.3. Conditionals

Conditional if

BÆNG 2.8

```
if condition:
    indented code
```

Conditional if JSON equivalent

BÆNG-JSON 2.9

```
["if", "condition", [
    /* code */
]]
```

The conditional `if` statement evaluates `condition` and executes the associated indented block only if the result evaluates to `True`. Conditions typically consist of comparison or logical expressions. If the condition evaluates to `False`, the indented block is skipped and execution continues with the next statement following the block. Conditionals enable selective execution of code, allowing programs to react to intermediate values, sample positions, or user-defined parameters during processing.

2.2.4. Loops

While loop

BÆNG 2.10

```
while condition:
    indented code
```

While loop JSON equivalent

BÆNG-JSON 2.11

```
["while", "condition", [
    /* code */
]]
```

The while loop repeatedly evaluates `condition` and executes the indented block for as long as the condition remains `true`. The condition is checked before each iteration. If the condition becomes `false`, execution continues after the loop. While loops are typically used for iterative calculations or state-dependent processing that cannot be expressed as a single-pass operation. Care must be taken to ensure that the loop condition eventually becomes false, otherwise execution will continue indefinitely.

In BÆNG, there is no dedicated for loop construct. Iterative operations that would typically use a for loop must instead be implemented using `while` loops. Common workarounds include initializing a counter variable and incrementing it manually within a while loop.

For-equivalent while loop

BÆNG 2.12

```
cnt = 0
while cnt < iterations:
    [indented code]
    cnt = cnt + 1
```

For-equivalent while loop JSON equivalent

BÆNG-JSON 2.13

```
["while", "cnt < iterations", [
    /* code */
    ["define", "cnt", "cnt + 1"]
]]
```

2.2.5. Console output

Print

BÆNG 2.14

```
print(expr)
```

Print JSON equivalent

BÆNG-JSON 2.15

```
["print", "expr"]
```

The `print` statement evaluates `expr` and outputs its value to the standard output. This command is primarily intended for debugging, inspection of intermediate values, and verification of program behavior during development. Printing does not modify the impulse response or affect program state beyond producing output for the user.

2.2.6. Impulse Response Manipulation

Set sample

BÆNG 2.16

```
setSample(index, value)
```

Set sample JSON equivalent

BÆNG-JSON 2.17

```
["setSample", index, value]
```

The `setSample` command writes the specified `value` to the impulse response at position `index`. The index refers to a discrete sample location within the IR buffer. It does not have to lie within the range of the impulse response. If an index outside of the defined impulse response is given, the command is ignored without raising an error. Otherwise, writing to a sample overwrites any existing value at that position. Passing floating-point indices is also possible. In this case the index gets rounded to the nearest integer index. Any expression can be used as `index` or `value`. This command provides direct, low-level access to the impulse response data and is typically used inside user functions to generate or modify signal content during iteration.

Read sample

BÆNG 2.18

```
readSample(index)
```

Read sample JSON equivalent

BÆNG-JSON 2.19

```
["readSample", index]
```

Deprecated. Use `IR[...]` instead.

The `readSample` command returns the value currently stored at the specified sample `index` in the impulse response. This allows previously written samples to be inspected or used in further calculations. Similar to `setSample`, indices outside of the defined impulse response can be passed, in which case the command returns 0. Also, floating-point indices are rounded to the nearest integer index. Reading samples enables feedback-like behavior, accumulation processes, or conditional processing based on existing IR content. The `readSample` command can be used as a regular complete expression for assignments, function arguments or conditions. It is **not** possible to combine this command with other operators to extend the expression, e.g. `readSample(2) + 1`.

Read sample in expression

BÆNG 2.20

```
IR[index]
```

Read sample in expression JSON equivalent

BÆNG-JSON 2.21

```
"IR[index]"
```

`IR[...]` is not a command on its own, but can be used as part of an expression or as a complete expression. Accessing sample values of the impulse response is identical to `readSample`. Float and out of range indices are supported. It is recommended to always use this command instead of `readSample`. An exception might be for better code readability when assigning the sample value directly to a variable.

2.2.7. Exporting the Impulse Response

Export

BÆNG 2.22

```
export(filename)
```

Export JSON equivalent

BÆNG-JSON 2.23

```
["export", filename]
```

The `export` command writes the current state of the impulse response to the specified output file. The filename determines the file location. The format is always a 16-bit PCM waveform audio file. Exporting finalizes the generated IR data and makes it available outside the execution environment. This command is usually used to save various states and versions of the processed IR throughout the program. It is automatically executed at the end of the main program once all processing steps have been completed. The final export is saved at the path given at the impulse response initialization (see BÆNG 2.1).

2.3. Expressions

Expressions are code snippets, that describe an arithmetic and/or boolean operation, which inherits a certain numerical value, a boolean value or a string. They cannot exist on their own, but must always be written in combination with another statement or command (see Section 2.2). In general, most python supported expressions are available. An expression must not span multiple code lines.

Expressions are evaluated from left to right unless operator precedence requires otherwise. Logical operators use short-circuit evaluation: in `A and B`, `B` is evaluated only if `A` evaluates to `True`; in `A or B`, `B` is evaluated only if `A` evaluates to `False`. Operator precedence and associativity follow Python rules. Parentheses may be used to explicitly control evaluation order.

2.3.1. Numerical Expressions

Numerical expressions support **arithmetical operators**:

Addition	BÆNG 2.24	Subtraction	BÆNG 2.25
<code>4 + 2</code>		<code>4 - 2</code>	
Multiplication	BÆNG 2.26	Division	BÆNG 2.27
<code>4 * 2</code>		<code>4 / 2</code>	
Exponentiation	BÆNG 2.28	Modulus	BÆNG 2.29
<code>4 ** 2</code>		<code>7 % 3</code>	
Floor division			BÆNG 2.30
<code>7 // 3</code>			

Numbers may be integers or floating-point values. When integer and floating-point values are combined, the result is a floating-point value. Division (`/`) always produces a floating-point result, while floor division (`//`) produces the largest integer less than or equal to the result.

2.3.2. Boolean Expressions

Boolean expressions evaluate to either `True` or `False`. They support **comparison operators**:

Equal	BÆNG 2.31	Not equal	BÆNG 2.32
<code>4 == 2</code>		<code>4 != 2</code>	
Greater than	BÆNG 2.33	Less than	BÆNG 2.34
<code>4 > 2</code>		<code>4 < 2</code>	
Greater than or equal to	BÆNG 2.35	Less than or equal to	BÆNG 2.36
<code>4 >= 2</code>		<code>4 <= 2</code>	

and **logical operators**:

And

BÆNG 2.37

`True and False`**Or**

BÆNG 2.38

`True or False`**Not**

BÆNG 2.39

`not True`

Logical operators return boolean values. Numerical and boolean expressions may be combined where meaningful. Boolean values may participate in numerical expressions according to Python semantics (`True` behaves as `1`, `False` as `0`).

2.3.3. Operator Precedence and Parentheses

Parentheses, **variable** integration and **operator precedence** is identical to python. Parentheses override precedence and are recommended for clarity in complex expressions.

Example expression

BÆNG 2.40

`not ((my_var * 2.5 + 3) > 10 and (my_var % 2 != 0)) or my_other_var == False`

2.3.4. String Expressions

Strings are first-class values and may be stored in variables, passed to functions, and used wherever a string value is expected. Strings are primarily used with `export(path: str)` and `print(content: any)`, but all supported string operations evaluate as normal expressions. The following Python-compatible string operations are supported:

String Slicing

BÆNG 2.41

`"Hello World"[2:5]`**String concatenation**

BÆNG 2.42

`"Hello" + " " + "World"`**Formatted Strings**

BÆNG 2.43

`f"Error at {error_location + 1}"`**Escape Characters**

BÆNG 2.44

`"Hello\nWorld"`

Common Python string methods are supported.

2.3.5. Built-in Functions

Also supported are **python built-in functions**, like `int()`, `float()`, `bool()`, `abs()`, etc. and the whole **numpy** package, accessible as `np.`:

Numpy

BÆNG 2.45

`np.ceil(a + np.pi)`**Built-in functions**

BÆNG 2.46

`bool(2 > abs(int(True) - 3))`

2.4. Conditions

Conditions are expressions, which can be evaluated as a boolean value with `bool()`. In this sense, every expression, mentioned in Section 2.3, can be used as a condition, including strings. Of course most of them are not suited as conditions, since they describe a fixed value. BÆNG 2.40 could be a logically sensible condition.

3. Functions and Execution Model

3.1. Overview

In BÆNG, user-defined functions describe operations that are applied to the impulse response rather than explicitly controlling iteration. The runtime is responsible for traversing the impulse response sample by sample. When a function is called, its body is executed once for every sample position in the impulse response unless restricted by conditional statements. This execution model allows users to focus on signal-processing logic while iteration and indexing are handled internally by the interpreter.

3.2. Function Definition and Parameters

A function is defined using the `func` keyword followed by a function name and an optional parameter list. Parameters act as local variables whose values are provided when the function is called. These variables exist only within the scope of the function and do not affect variables outside the function unless explicitly written to shared program state, such as the impulse response.

Function parameters are evaluated before execution begins. This ensures that each function invocation operates on a consistent set of input values throughout its iteration over the impulse response.

3.3. Iteration Over the Impulse Response

Unlike conventional programming languages where loops explicitly control iteration, BÆNG implicitly iterates over the impulse response whenever a function is executed. Internally, the interpreter performs a loop over all samples from index `0` to the final sample index determined by the initialized IR length. During each iteration, the function body is evaluated once.

This design ensures that signal-processing logic remains concise and reduces the need for manual indexing logic in user code. Explicit loops such as `while` are therefore primarily used for secondary iteration tasks, such as generating multiple reflections or performing repeated calculations within a single sample step.

During execution, the interpreter automatically maintains the variable `SAMPLEPOS`. This variable represents the current sample index being processed and is updated before each function iteration step. It allows functions to access or modify the impulse response relative to the current processing position. `SAMPLEPOS` is read-only and cannot be modified by user code. Typical use cases include:

- triggering events at specific sample positions,
- reading previously written samples,
- writing values to the impulse response
- generating time-dependent behavior,
- implementing delay-based or recursive structures.

3.4. Side Effects and Data Modification

Functions in BÆNG primarily operate through side effects. Commands such as `setSample()` modify the impulse response directly, and these modifications persist across subsequent iterations and function calls. Because execution proceeds sequentially over sample positions, earlier writes influence later processing steps. Care must therefore be taken when reading and writing samples within the same function, as execution order determines the resulting signal structure.

3.5. Function Calls and Execution Order

Function calls in the main code are executed in the order they appear. Each call completes its full iteration over the impulse response before the next statement is evaluated. If multiple functions modify the impulse response, later function calls operate on the already modified state produced by earlier calls. This sequential execution model allows complex processing pipelines to be constructed by chaining multiple function calls, where each stage progressively transforms the impulse response.

4. Complete Example

The following code example (BÆNG 4.1) demonstrates a simple room-reverb simulation. The simulation takes basic parameters: `pre_delay`, `decay`, `size`, `density`, `recursion_number` and `fs`, which are passed to the main function `RoomSim`. It internally calls the auxiliary function `RecursiveReflections`. It does not provide frequency dependent characteristics.

complete_example.baeng

BÆNG 4.1

```
set IR: (44100, 2, "outputDecay20.wav")

func RoomSim(pre_delay, decay, size, density, recursion_number, fs):
    if SAMPLEPOS == 0:
        setSample(int(round(pre_delay * fs)), 1)

    n = 0
    while n < recursion_number:
        print(f"Iteration {n + 1}...")
        RecursiveReflections(decay, size, density, fs)
        n = n + 1

func RecursiveReflections(decay, size, density, fs):
    current_peak = readSample(SAMPLEPOS)

    if abs(current_peak) > 0.001:

        reflection_sample_range = int(round(size * fs))
        reflection_number = int(round(density * size))

        i = 0
        while i < reflection_number:
            reflection_sample = SAMPLEPOS + np.random.randint(reflection_sample_range)
            current_reflection_value = readSample(reflection_sample)
            exp_factor = np.exp(-decay * (reflection_sample - SAMPLEPOS) / fs)
            reflection_value = - current_peak * exp_factor + current_reflection_value

            setSample(reflection_sample, reflection_value)
            i = i + 1

RoomSim(0.01, 20, 2, 10, 20, 44100)
```

4.1. Initialization of the Impulse Response

Before running the simulation, the impulse response buffer is initialized (see BÆNG 4.2). This creates an impulse response of 2 s at a sampling rate of 44.1 kHz and specifies the output file `"output.wav"`. Both duration and sampling rate can be freely adjusted.

IR Initialization

BÆNG 4.2

```
set IR: (44100, 2, "outputDecay20.wav")
```

4.2. Room Simulation Function

The `RoomSim` function is the main entry point of the simulation. It places an initial excitation into the impulse response and performs a configurable number of recursive reflection passes. Each pass enriches the impulse response with additional reflections, creating a progressively denser and more realistic reverberation tail.

It receives the following parameters:

pre_delay: *int* / *float*

The time before the first reflection in seconds.

decay: *int* / *float*

An exponential decay factor for reflections (see Section 4.3).

size: *int* / *float*

The length in seconds of the time window, in which each impulse creates new reflections.

density: *int* / *float*

The average density of reflections in reflections per second, that get created each (recursive) iteration (see Section 4.3).

recursion_number: *int*

The number of recursive reflection passes. This affects the density of the reverb.

fs: *int*

The sampling frequency of the impulse response.

RoomSim Function

BÆNG 4.3

```
func RoomSim(pre_delay, decay, size, density, recursion_number, fs):
    if SAMPLEPOS == 0:
        setSample(int(round(pre_delay * fs)), 1)

    n = 0
    while n < recursion_number:
        print(f"Iteration {n + 1}...")
        RecursiveReflections(decay, size, density, fs)
        n = n + 1
```

`RoomSim` is executed once for every sample position in the impulse response. However, its main logic is only triggered at the beginning of the IR, by checking if `SAMPLEPOS == 0`. This is a recommended solution in BÆNG to execute global initialization in functions only during the first iteration. It mimics the use of conventional functions in other programming languages, e.g. python.

The function computes the sample index corresponding to the given `pre_delay` and sets the impulse response to a value of 1 at that position. This acts as an initial excitation of the simulated room. Apart from this initial impulse, `RoomSim` itself does not manipulate samples of the impulse response.

The function then enters a loop that runs `recursion_number` times. In each iteration, it prints out the number of the current iteration starting at 1 with a formatted string, e.g. `Iteration 1...`. Then, it calls `RecursiveReflections`, which scans each sample position and generates new reflections based on the existing amplitude distribution. The required arguments for `RecursiveReflections` are passed from the given arguments of `RoomSim`.

4.3. Recursive Reflections Function

The `RecursiveReflections` function generates new reflections based on the amplitude of the current sample. If the absolute value at the current sample is large enough, the function computes a number of new reflection positions and writes dampened copies of the current peak into the impulse response.

decay: *int* / *float*

An exponential decay factor in s^{-1} for reflections in the form of $e^{-\lambda \cdot \Delta t}$, where the decay is λ and the delay between the reflection and the impulse, creating the reflection, is Δt in seconds.

size: *int* / *float*

The length in seconds of the time window, in which each impulse creates new reflections.

density: *int* / *float*

The average density of reflections in reflections per second, that get created each (recursive) iteration. It is multiplied by the size and rounded to determine the number of reflections generated.

fs: *int*

The sampling frequency of the impulse response.

RecursiveReflections Function

BÆNG 4.4

```
func RecursiveReflections(decay, size, density, fs):
    current_peak = readSample(SAMPLEPOS)

    if abs(current_peak) > 0.001:

        reflection_sample_range = int(round(size * fs))
        reflection_number = int(round(density * size))

        i = 0
        while i < reflection_number:
            reflection_sample = SAMPLEPOS + np.random.randint(reflection_sample_range)
            current_reflection_value = readSample(reflection_sample)
            exp_factor = np.exp(-decay * (reflection_sample - SAMPLEPOS) / fs)
            reflection_value = - current_peak * exp_factor + current_reflection_value

            setSample(reflection_sample, reflection_value)
            i = i + 1
```

`RecursiveReflections` is called repeatedly during the simulation and is responsible for generating the actual reverberation structure.

The function begins by reading the amplitude at the current sample position: `readSample(SAMPLEPOS)`. Only if the absolute value at this sample exceeds a small threshold (0.001) does the function generate reflections. This prevents unnecessary computation, thus decreases computation time and avoids propagating negligible noise.

Next, the reflection parameters are determined. `reflection_sample_range` defines how far into the future reflections may be placed. It is proportional to the room size. `reflection_number` determines how many reflections will be generated. It scales with both `density` and `size`.

For each reflection, a random offset within the allowed range is added to `SAMPLEPOS`, producing a future sample index. The current value at that position is read, allowing reflections to accumulate over multiple passes. A new reflection value is calculated based on the current value at that position and an inverted, exponentially decaying copy of the value at the current position (see Equation (4.1)).

$$x[n] = -x[n] \cdot \exp\left(-\lambda \cdot \frac{\Delta n}{f_s}\right) + x[n + \Delta n] \quad (4.1)$$

The resulting value is written back into the impulse response using `setSample`.

Because reflections are directly written into future positions, later function iterations from the same call, as well as future function calls will read the newly created peaks and generate further reflections until the absolute value falls below the threshold of 0.001.

4.4. Running the Simulation

The simulation can be run by calling the `RoomSim` function. In this example the main code only consists of this function call with the following parameters:

- 10 ms pre-delay
- decay factor $20 \frac{1}{s}$
- room size 2 s
- density $10 \frac{1}{s}$
- 20 recursion passes
- sampling rate 44.1 kHz as defined in BÆNG 4.2

Complete example

BÆNG 4.5

```
RoomSim(0.01, 20, 2, 10, 20, 44100)
```

The resulting impulse response is similar to a mid sized bathroom.

5. General Coding Conventions

Functions may be defined at any position in the main code as long as they appear at the outermost indentation level (i.e., not inside loops or conditionals). Regardless of their position, all functions are globally visible and may be called from anywhere in the program. This means a function may be defined at the end of the file while being called at the beginning.

For readability and maintainability, it is recommended to place all function definitions before the main execution section.

Because a function is executed once for every sample of the impulse response, it is often beneficial to avoid recomputing values that do not depend on the current sample position. Such values can be stored in global variables and initialized during the first iteration of the function by checking `SAMPLEPOS == 0`. This technique can also be used to call other functions only once from within a function.

When accessing the same sample value multiple times within a single function iteration, it is more efficient to read the value once using `readSample` and store it in a variable. The `readSample` command is generally faster than repeated `IR[...]` indexing because it performs a direct lookup without the additional expression-parsing overhead. Its only limitation is that it cannot be used directly inside expressions and must therefore be assigned to a variable first. Using a stored variable avoids repeated array accesses, improves performance, and still allows the value to be freely combined with other expressions.